

Dataset Description:

The dataset used in this project is a Starbucks Customer Ordering dataset. It contains transactional data including order details, customer demographics, store information, and product categories.

Key tables used:

- orders: Contains order-level details such as order_date, total_spend, fulfillment_time, and satisfaction
- customer: Contains customer demographics like age group, gender, and rewards membership
- store: Contains store-level details such as region and location type
- cart_details: Contains cart-level data like cart size, food items, and drink category
- drink_category_lookup: Maps drink category IDs to actual drink names

The dataset contains over 100,000 records, making it suitable for advanced SQL analysis and performance optimization.

Link for Dashboard: [Starbucks Final Dashboard](#)

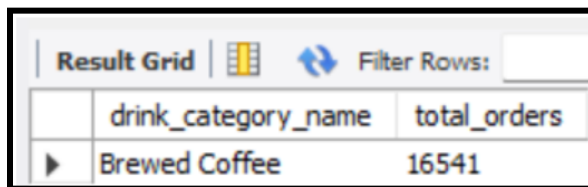
1. Basic Queries

1.1 Domain Question: Which drink is the least popular among the customers?

SQL Code:

```
SELECT
    d.drink_category_name,
    COUNT(*) AS total_orders
FROM cart_details c
JOIN drink_category_lookup d
    ON c.drink_category_id = d.drink_category_id
GROUP BY d.drink_category_name
ORDER BY total_orders ASC
LIMIT 1;
```

Result Snapshot:



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with two columns: 'drink_category_name' and 'total_orders'. The first and only row shown is 'Brewed Coffee' with a total of 16,541 orders. Above the table, there are icons for a grid, a chart, and a refresh button, along with a 'Filter Rows:' input field.

drink_category_name	total_orders
Brewed Coffee	16541

Insights:

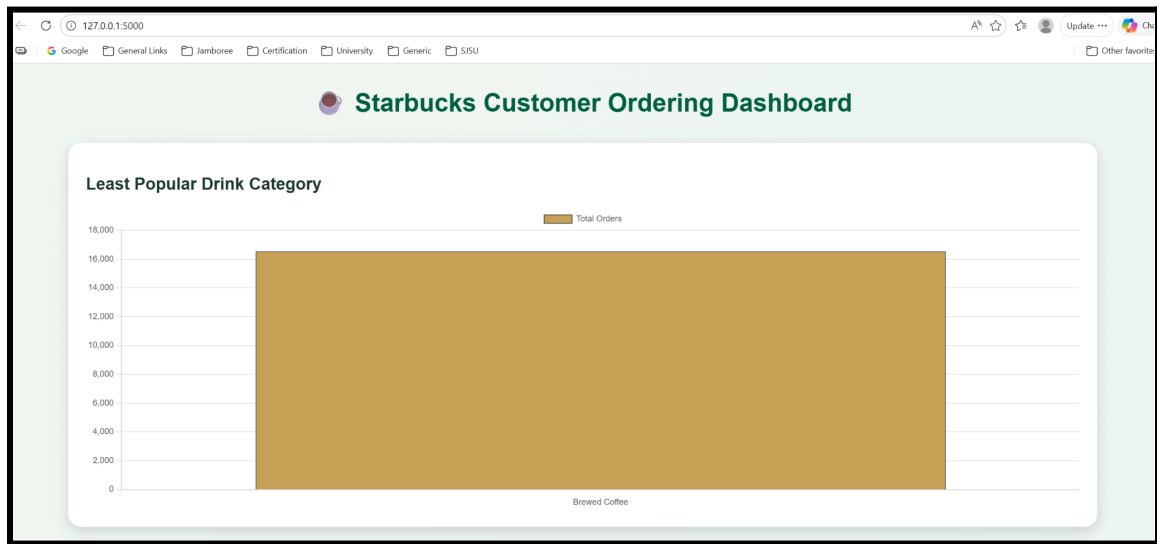
This query identifies the least popular drink category based on the lowest number of orders. From the result, “Brewed Coffee” has the minimum number of orders (16,541), making it the least preferred option among customers. This insight is important because it helps the business understand changing customer preferences.

It suggests that customers may be shifting toward more customized or specialty drinks rather than basic brewed coffee.

Starbucks can use this information to adjust its menu offerings, promotions, or product placement to better align with customer demand.

Additionally, this trend may indicate an opportunity to innovate or promote traditional products through marketing strategies or bundling offers.

Visualization:

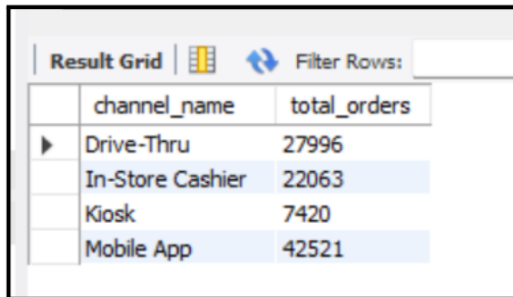


1.2 Domain Question: Count total order by Channel This tells you how many orders came from: Drive-Thru, Mobile App, Kiosk, In-Store Cashier

SQL Code:

```
SELECT
    c.channel_name,
    COUNT(o.order_id) AS total_orders
FROM orders o
JOIN channel_lookup c
    ON o.channel_id = c.channel_id
GROUP BY c.channel_name;
```

Result Snapshot:



The screenshot shows a database interface with a 'Result Grid' tab selected. The grid displays the results of the SQL query, showing the channel name and the total number of orders for each channel. The data is as follows:

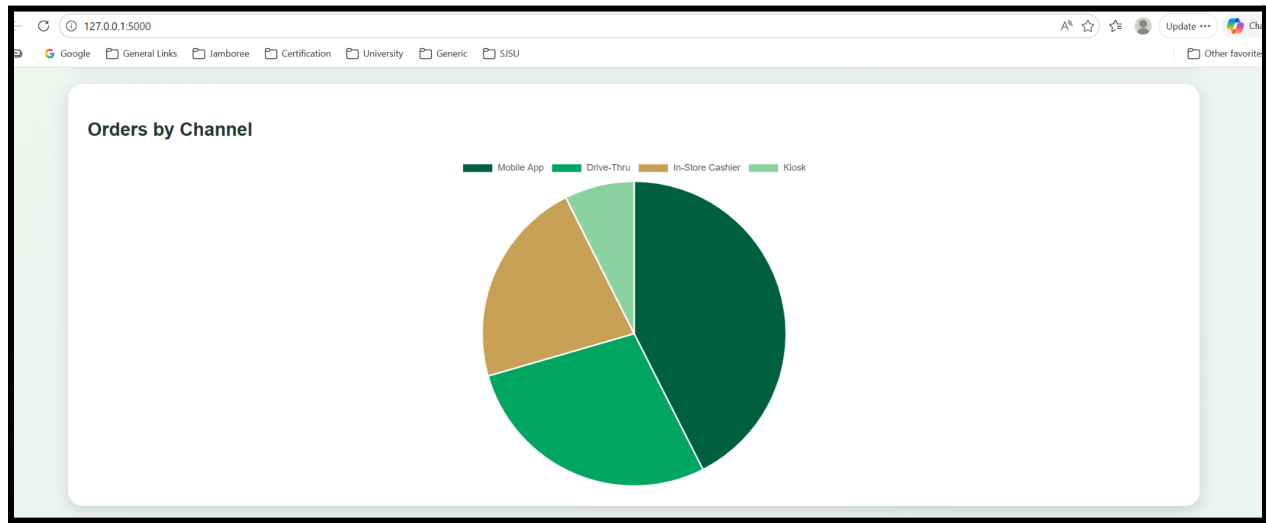
	channel_name	total_orders
▶	Drive-Thru	27996
	In-Store Cashier	22063
	Kiosk	7420
	Mobile App	42521

Insights:

This query shows that the Mobile App is the most popular ordering channel, followed by Drive-Thru and In-Store Cashier, while Kiosk is the least used.

This indicates that customers prefer quick and convenient digital ordering options, highlighting the importance of mobile-based services.

Visualization:

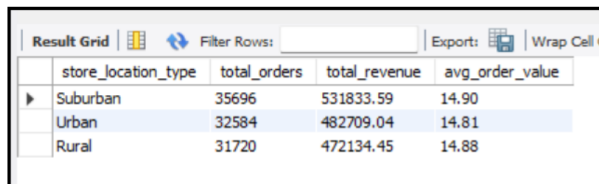


1.3 Domain Question: Which store location type generates the highest revenue?

SQL Code:

```
SELECT
    s.store_location_type,
    COUNT(o.order_id) AS total_orders,
    ROUND(SUM(o.total_spend), 2) AS total_revenue,
    ROUND(AVG(o.total_spend), 2) AS avg_order_value
FROM store s
JOIN orders o
    ON s.store_id = o.store_id
GROUP BY s.store_location_type
ORDER BY total_revenue DESC;
```

Result Snapshot:



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with four columns: 'store_location_type', 'total_orders', 'total_revenue', and 'avg_order_value'. The data is sorted by 'total_revenue' in descending order. The rows are: Suburban (35696 orders, 531833.59 revenue, 14.90 avg), Urban (32584 orders, 482709.04 revenue, 14.81 avg), and Rural (31720 orders, 472134.45 revenue, 14.88 avg). The 'Urban' row is highlighted with a blue background.

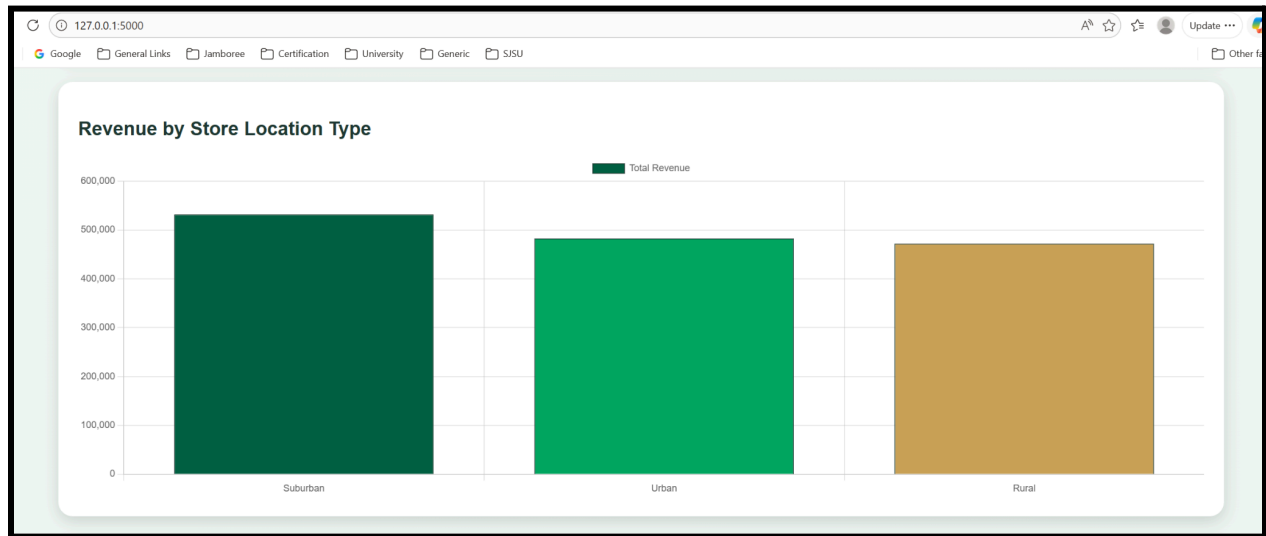
store_location_type	total_orders	total_revenue	avg_order_value
Suburban	35696	531833.59	14.90
Urban	32584	482709.04	14.81
Rural	31720	472134.45	14.88

Insights:

Suburban stores generate the highest revenue and also have the highest number of orders, making them the most profitable location type.

This suggests that expanding suburban store formats could maximize business revenue.

Visualization:

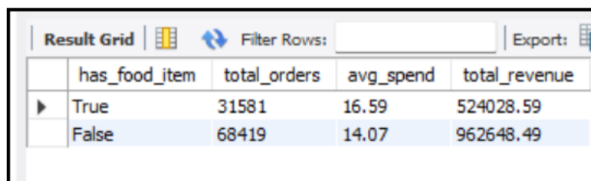


1.4 Domain Question: Do orders with food items have higher average spend than drink-only orders?

SQL Code:

```
SELECT
    cd.has_food_item,
    COUNT(o.order_id) AS total_orders,
    ROUND(AVG(o.total_spend), 2) AS avg_spend,
    ROUND(SUM(o.total_spend), 2) AS total_revenue
FROM orders o
JOIN cart_details cd
    ON o.order_id = cd.order_id
GROUP BY cd.has_food_item
ORDER BY avg_spend DESC;
```

Result Snapshot:



The screenshot shows a database interface with a 'Result Grid' tab. It contains a table with 5 columns: 'has_food_item', 'total_orders', 'avg_spend', and 'total_revenue'. There are two rows of data. The first row is for 'True' (has food item), showing 31581 total orders, an average spend of 16.59, and a total revenue of 524028.59. The second row is for 'False' (no food item), showing 68419 total orders, an average spend of 14.07, and a total revenue of 962648.49. The interface also includes a 'Filter Rows' button and an 'Export' button.

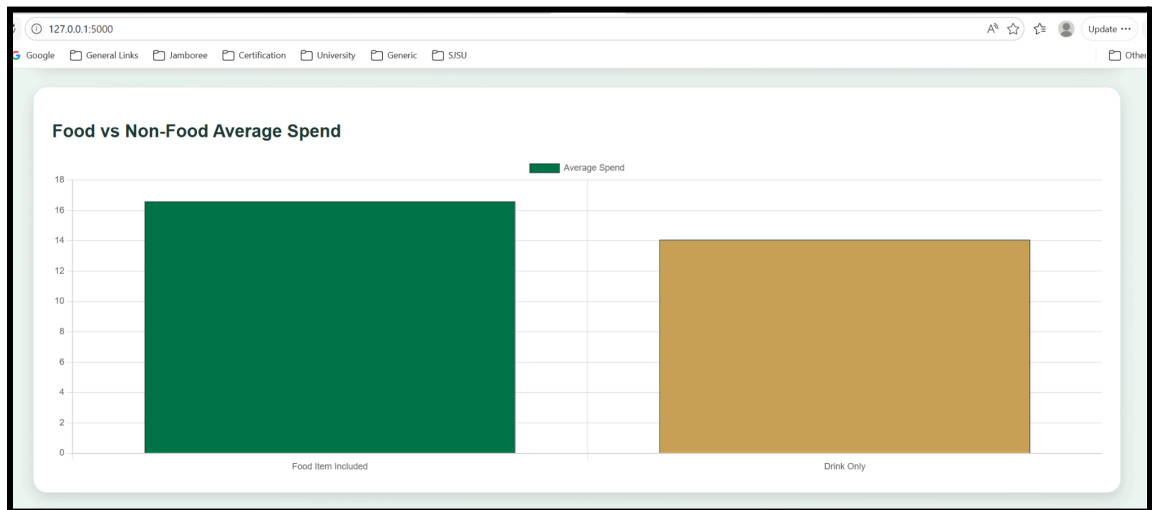
	has_food_item	total_orders	avg_spend	total_revenue
▶	True	31581	16.59	524028.59
	False	68419	14.07	962648.49

Insights:

Orders with food items have a higher average spend (\$16.59) compared to drink-only orders (\$14.07).

However, drink-only orders contribute more to total revenue due to a significantly higher number of orders. This suggests that while food increases per-order value, drinks drive overall sales volume.

Visualization:



1.5 Domain Question: Which region has the highest average fulfillment time?

SQL Code:

```
SELECT
    s.region,
    COUNT(o.order_id) AS total_orders,
    ROUND(AVG(o.fulfillment_time_min), 2) AS avg_fulfillment_time,
    ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
FROM store s
JOIN orders o
    ON s.store_id = o.store_id
GROUP BY s.region
ORDER BY avg_fulfillment_time DESC
LIMIT 10;
```

Result Snapshot:

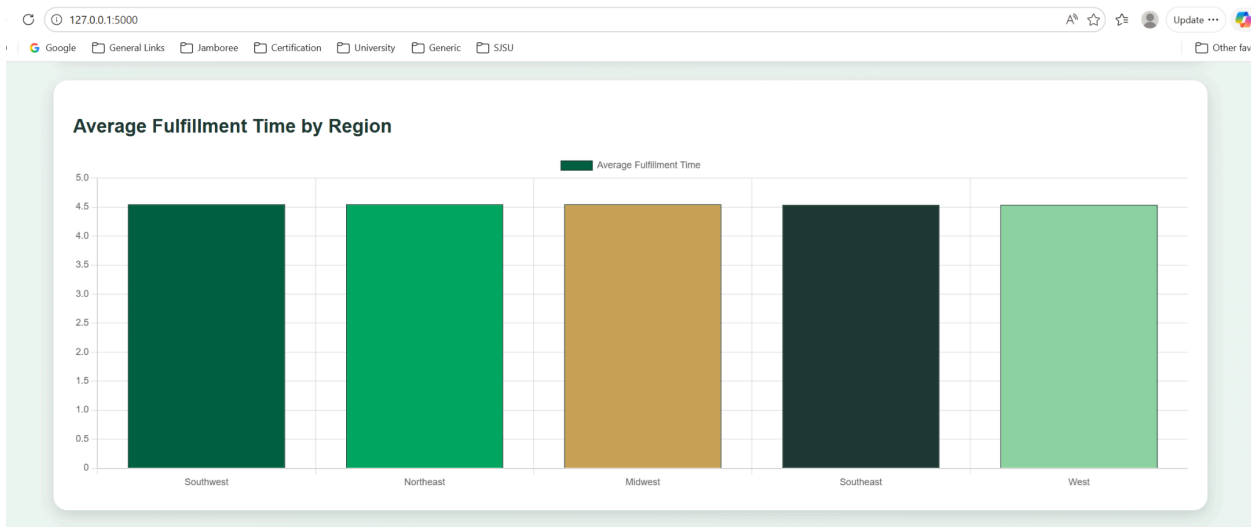
	region	total_orders	avg_fulfillment_time	avg_satisfaction
►	Southwest	19663	4.55	3.70
	Northeast	18036	4.55	3.67
	Midwest	19470	4.55	3.67
	Southeast	20223	4.54	3.70
	West	22608	4.54	3.69

Insights:

The average fulfillment time is nearly the same across all regions, ranging between 4.54 and 4.55 minutes.

This indicates that service performance is consistent across regions, with no significant delays in any specific area. There is no major regional bottleneck affecting order fulfillment.

Visualization:



2. Advance Query

2.1 Domain Question: Identify the years present in the dataset, find the top-selling month within each year, calculate the average customer spend for that month, and determine the most preferred drink category.

SQL Code:

```
WITH monthly_sales AS (
    SELECT
        YEAR(o.order_date) AS sales_year,
        MONTH(o.order_date) AS sales_month,
        MONTHNAME(o.order_date) AS month_name,
        SUM(o.total_spend) AS total_sales,
        AVG(o.total_spend) AS avg_customer_spend
    FROM orders o
    GROUP BY
        YEAR(o.order_date),
        MONTH(o.order_date),
        MONTHNAME(o.order_date)
),
ranked_months AS (
    SELECT
        sales_year,
        sales_month,
        month_name,
        total_sales,
        avg_customer_spend,
        RANK() OVER (
            PARTITION BY sales_year
            ORDER BY total_sales DESC
        ) AS month_rank
    FROM monthly_sales
),
top_month_per_year AS (
    SELECT
        sales_year,
        sales_month,
        month_name,
        total_sales,
        avg_customer_spend
    FROM ranked_months
    WHERE month_rank = 1
),
```

```

drink_counts AS (
  SELECT
    YEAR(o.order_date) AS sales_year,
    MONTH(o.order_date) AS sales_month,
    d.drink_category_name,
    COUNT(*) AS total_orders,
    RANK() OVER (
      PARTITION BY YEAR(o.order_date), MONTH(o.order_date)
      ORDER BY COUNT(*) DESC
    ) AS drink_rank
  FROM orders o
  JOIN cart_details c
    ON o.order_id = c.order_id
  JOIN drink_category_lookup d
    ON c.drink_category_id = d.drink_category_id
  GROUP BY
    YEAR(o.order_date),
    MONTH(o.order_date),
    d.drink_category_name
),
top_drink_per_month AS (
  SELECT
    sales_year,
    sales_month,
    drink_category_name,
    total_orders
  FROM drink_counts
  WHERE drink_rank = 1
)
SELECT
  t.sales_year AS year,
  t.month_name AS top_selling_month,
  t.total_sales,
  t.avg_customer_spend,
  d.drink_category_name AS most_preferred_drink
FROM top_month_per_year t
JOIN top_drink_per_month d
  ON t.sales_year = d.sales_year
  AND t.sales_month = d.sales_month
ORDER BY t.sales_year;

```

Result Snapshot:

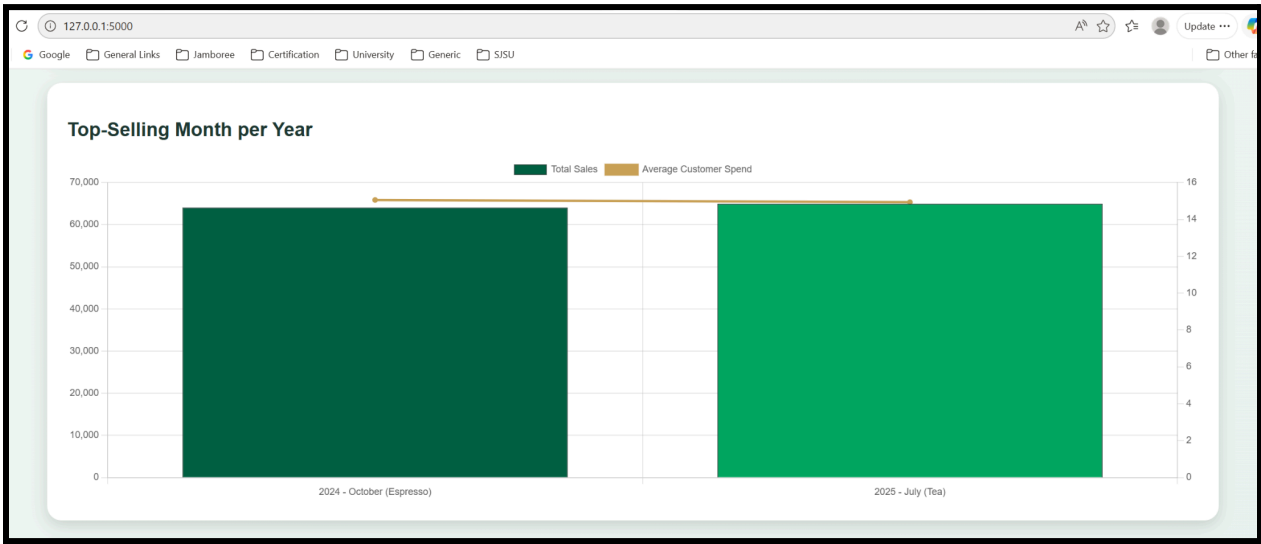
Result Grid				
Filter Rows:		Export:		Wrap Cell Content:
year	top_selling_month	total_sales	avg_customer_spend	most_preferred_drink
2024	October	64027.13	15.051041	Espresso
2025	July	64933.88	14.927329	Tea

Insights:

October (2024) and July (2025) are the top-performing months, with Espresso and Tea as the most preferred drinks respectively.

This indicates seasonal variations in both sales and customer preferences, suggesting that marketing strategies and inventory planning should adapt based on time of year.

Visualization:



2.2 Domain Question: Identify the top 10, which Starbucks stores generated revenue higher than the average store revenue.

SQL Code:

```
SELECT
    s.store_id,
    s.region,
    s.store_location_type,
    ROUND(SUM(o.total_spend), 2) AS store_revenue
FROM store s
JOIN orders o
    ON s.store_id = o.store_id
GROUP BY
    s.store_id,
    s.region,
    s.store_location_type
HAVING SUM(o.total_spend) > (
    SELECT AVG(store_total)
    FROM (
        SELECT
            store_id,
            SUM(total_spend) AS store_total
        FROM orders
        GROUP BY store_id
    ) AS store_avg
)
ORDER BY store_revenue DESC
LIMIT 10;
```

Result Snapshot:

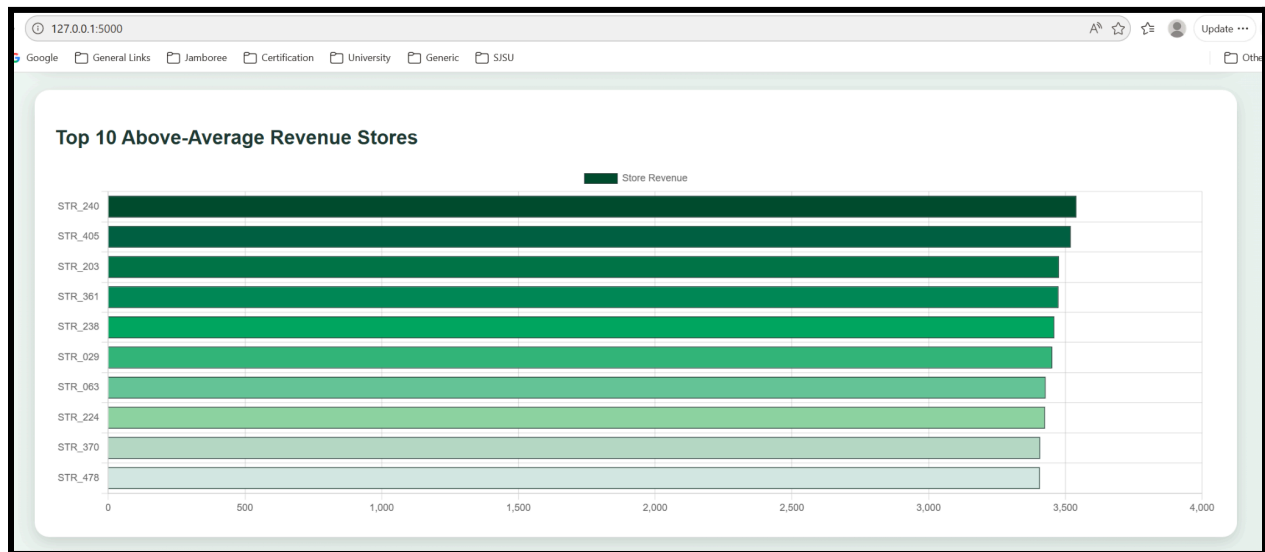
	store_id	region	store_location_type	store_revenue
►	STR_240	Northeast	Urban	3540.66
	STR_405	Southeast	Rural	3519.52
	STR_203	Southeast	Rural	3476.85
	STR_361	Northeast	Suburban	3474.82
	STR_238	Midwest	Suburban	3459.53
	STR_029	Midwest	Urban	3451.80
	STR_063	Southeast	Suburban	3427.44
	STR_224	Midwest	Rural	3425.38
	STR_370	West	Suburban	3407.50
	STR_478	Northeast	Urban	3406.58

Insights:

The top-performing stores generating above-average revenue are spread across multiple regions, with a strong presence in the Northeast and Southeast.

Both Urban and Suburban store types appear frequently among the top performers, indicating that high revenue is not limited to a single location type. These stores can serve as benchmarks to identify successful strategies in operations, location planning, and customer engagement.

Visualization:



2.3 Domain Question: Identify the top customers and classify them into High, Medium, or Low value segments based on total spending.

SQL Code:

```
WITH customer_spending AS (  
    SELECT  
        customer_id,  
        ROUND(SUM(total_spend), 2) AS total_spending  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT  
    customer_id,  
    total_spending,  
    CASE  
        WHEN total_spending >= 280 THEN 'High Value'  
        WHEN total_spending >= 270 THEN 'Medium Value'  
        ELSE 'Low Value'  
    END AS customer_segment  
FROM customer_spending  
ORDER BY total_spending DESC  
LIMIT 10;
```

Result Snapshot:

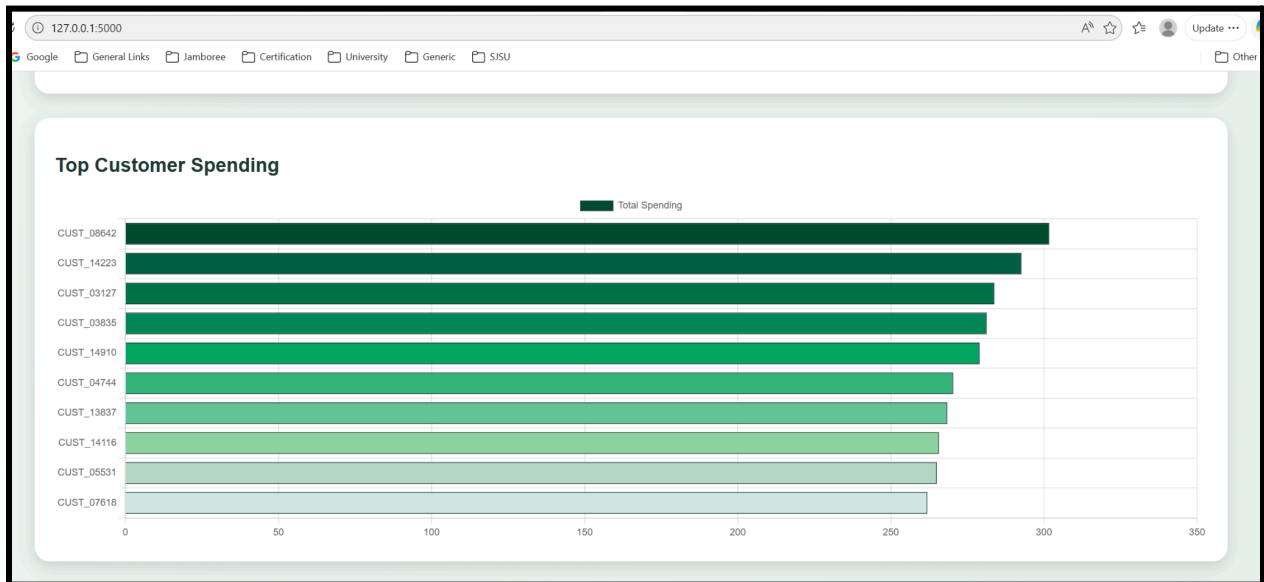
	customer_id	total_spending	customer_segment
▶	CUST_08642	301.68	High Value
	CUST_14223	292.64	High Value
	CUST_03127	283.81	High Value
	CUST_03835	281.27	High Value
	CUST_14910	278.99	Medium Value
	CUST_04744	270.36	Medium Value
	CUST_13837	268.42	Low Value
	CUST_14116	265.65	Low Value
	CUST_05531	265.03	Low Value
	CUST_07618	261.93	Low Value

Insights:

The top customers are primarily classified as High Value, indicating that a small group of customers contributes significantly to overall revenue.

Medium and Low Value customers appear less frequently among the top spenders, suggesting an opportunity to target them with promotions to increase their spending.

Visualization:



2.4 Domain Question: Identify which Starbucks stores rank in the top 3 for customer satisfaction within each region.

SQL Code:

```
WITH store_satisfaction AS (  
    SELECT  
        s.region,  
        s.store_id,  
        s.store_location_type,  
        ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction,  
        COUNT(o.order_id) AS total_orders  
    FROM store s  
    JOIN orders o  
        ON s.store_id = o.store_id  
    GROUP BY s.region, s.store_id, s.store_location_type  
)  
ranked_stores AS (  
    SELECT  
        region,  
        store_id,  
        store_location_type,  
        avg_satisfaction,  
        total_orders,  
        RANK() OVER (  
            PARTITION BY region  
            ORDER BY avg_satisfaction DESC  
        ) AS satisfaction_rank  
    FROM store_satisfaction  
)  
SELECT  
    region,  
    store_id,  
    store_location_type,  
    avg_satisfaction,  
    total_orders,  
    satisfaction_rank  
FROM ranked_stores  
WHERE satisfaction_rank <= 3  
ORDER BY region, satisfaction_rank;
```

Result Snapshot:

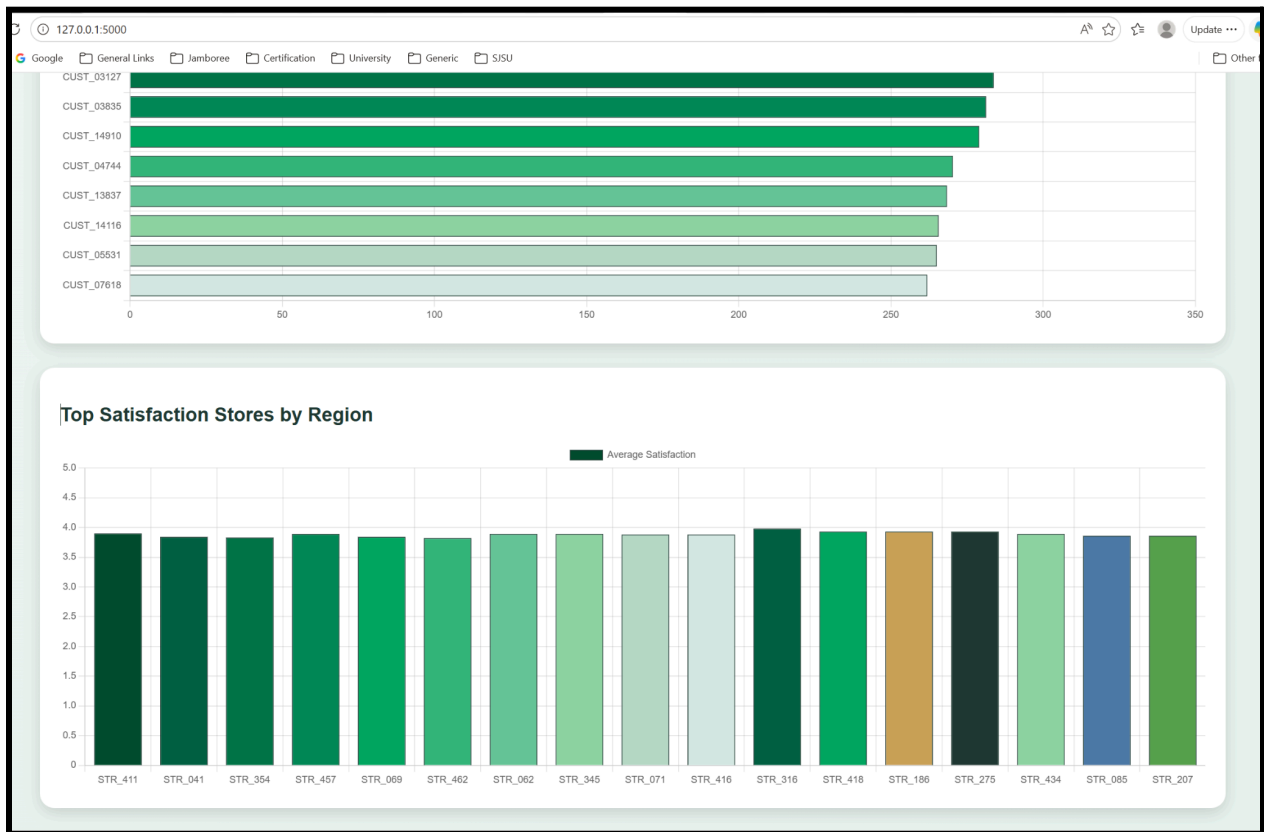
	region	store_id	store_location_type	avg_satisfaction	total_orders	satisfaction_rank
▶	Midwest	STR_411	Urban	3.90	201	1
	Midwest	STR_041	Urban	3.84	230	2
	Midwest	STR_354	Suburban	3.83	213	3
	Northeast	STR_457	Rural	3.89	211	1
	Northeast	STR_069	Urban	3.84	206	2
	Northeast	STR_462	Rural	3.82	216	3
	Southeast	STR_062	Rural	3.89	184	1
	Southeast	STR_345	Urban	3.89	195	1
	Southeast	STR_071	Urban	3.88	176	3
	Southeast	STR_416	Urban	3.88	207	3
	Southwest	STR_316	Rural	3.98	198	1
	Southwest	STR_418	Urban	3.93	203	2
	Southwest	STR_186	Suburban	3.93	178	2
	West	STR_275	Rural	3.93	190	1
	West	STR_434	Rural	3.89	212	2
	West	STR_085	Suburban	3.86	191	3
	West	STR_207	Suburban	3.86	176	3

Insights:

Top-performing stores in terms of customer satisfaction are identified within each region, ensuring a fair comparison across different geographic areas.

The results show that high satisfaction scores are consistently achieved across various store types (Urban, Rural, Suburban), indicating that good customer experience is driven more by service quality than location type. These top-ranked stores can serve as benchmarks to improve performance in lower-ranked locations.

Visualization:



2.5 Domain Question: Check whether indexing can improve query performance for order-ahead orders with longer fulfillment time.

SQL Code:

Before Index -

```
EXPLAIN
SELECT
  order_id,
  store_id,
  order_date,
  order_ahead,
  fulfillment_time_min,
  customer_satisfaction
FROM orders
WHERE order_ahead = 'True'
  AND fulfillment_time_min > 10
ORDER BY fulfillment_time_min DESC;
```

Create Index -

```
CREATE INDEX idx_orderahead_fulfillment
ON orders(order_ahead, fulfillment_time_min);
```

After Index -

```
EXPLAIN
SELECT
  order_id,
  store_id,
  order_date,
  order_ahead,
  fulfillment_time_min,
  customer_satisfaction
FROM orders
WHERE order_ahead = 'True'
  AND fulfillment_time_min > 10
ORDER BY fulfillment_time_min DESC;
```

Result Snapshot:

Before Index

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	orders	NULL	ALL	NULL	NULL	NULL	NULL	99651	3.33	Using where; Using filesort

After index

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	orders	10000	range	idx_orderahead_fulfillment	idx_orderahead_fulfillment	89	10000	1	100.00	Using index condition; Backward index scan

Insights:

Before adding the index, MySQL performed a full table scan (`type = ALL`) and scanned almost the entire orders table, around 99,651 rows. It also used `Using where` and `Using filesort`, meaning the database had to filter and sort the data manually.

After creating the composite index on `order\_ahead` and `fulfillment\_time\_min`, MySQL used the index (`type = range`) and scanned only 1 row. This shows that indexing significantly improved query efficiency by reducing unnecessary scanning and sorting.